

Entegris Architecture Document

Integration Reference Architecture

SAP Write Backs

Version 0.1

Contents

Document Introduction	3
Purpose of Document	3
Scope and Applicability	3
Intended Audience	4
Context	5
Why to use Integration Patterns	5
AS-IS SAP Integration Architecture Diagram	6
Architecture Principles	7
Enterprise Principles	7
Integration Principles	7
Security-by-Design Principles	7
Pattern Selection Matrix (Guidance)	8
Canonical SAP Integration Patterns	11
Synchronous API (Request/Reply)	11
Batch SFTP (File-based Extract/Load)	12
Batch API (Scheduled Extract/Load)	13
SAP IDoc / RFC / OData (SAP-native)	14
Asynchronous Pattern	15
Tips and Best Practices for Integration	18

Document Introduction

This document is to be used as a reference for SAP Write back integration patterns. It lists the SAP integration patterns, details on when to use and when not to use.

Purpose of Document

This document defines the Integration Reference Architecture. It provides architectural guardrails, standards, and patterns to guide the design and implementation of integrations across SAP and non-SAP systems while using Boomi as the middleware.

- The value of the pattern is **speed to execution** - leveraging what others have done before and quickly achieving business outcomes
- A pattern is best expressed as, **When to use, When Not to use** certain technologies/ approach, etc.
- This document is for designers and architects who need to integrate with other applications in their enterprise. This content is a distillation of many successful implementations.
- If implemented properly, these patterns enable you to get to production as fast as possible and have the most stable, scalable, and maintenance-free set of applications possible.
- As with all patterns, this content covers most integration scenarios, but not all.

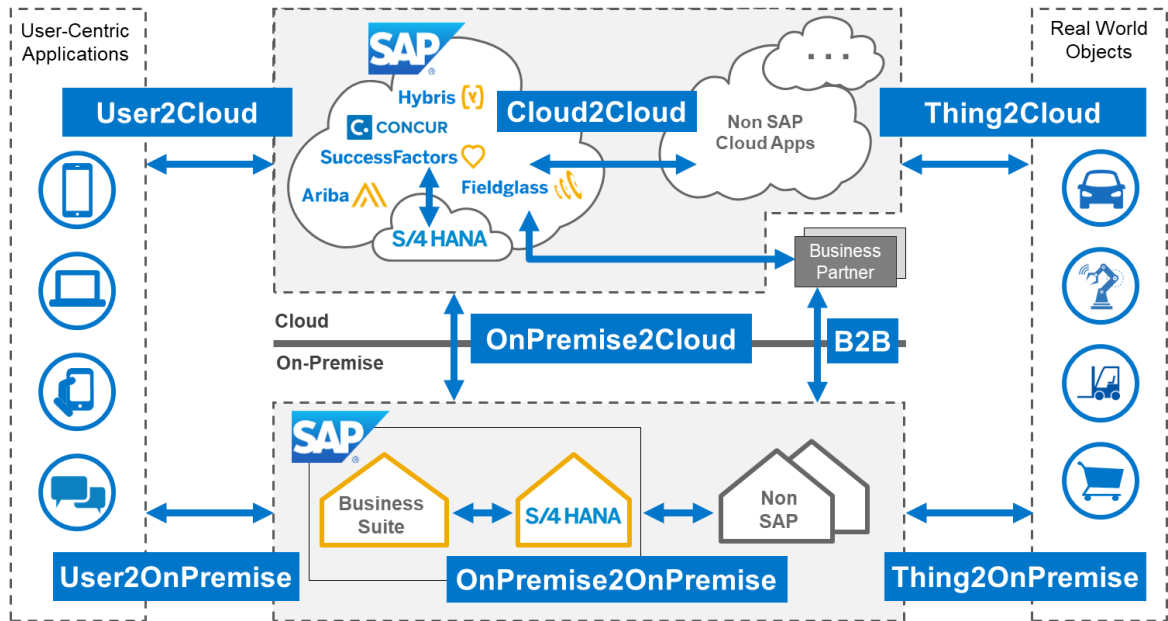
Scope and Applicability

The reference architecture applies to SAP to Non-SAP applications specifically for SAP write-backs. The scenarios considered are as follows:

- User to Cloud
- Cloud to Cloud
- On-Prem-to Cloud

Environments:

- Hybrid environments (on-prem, cloud, SaaS)



Intended Audience

- Solution Architects
- SAP Technical Leads
- Security and Operations Teams

Context

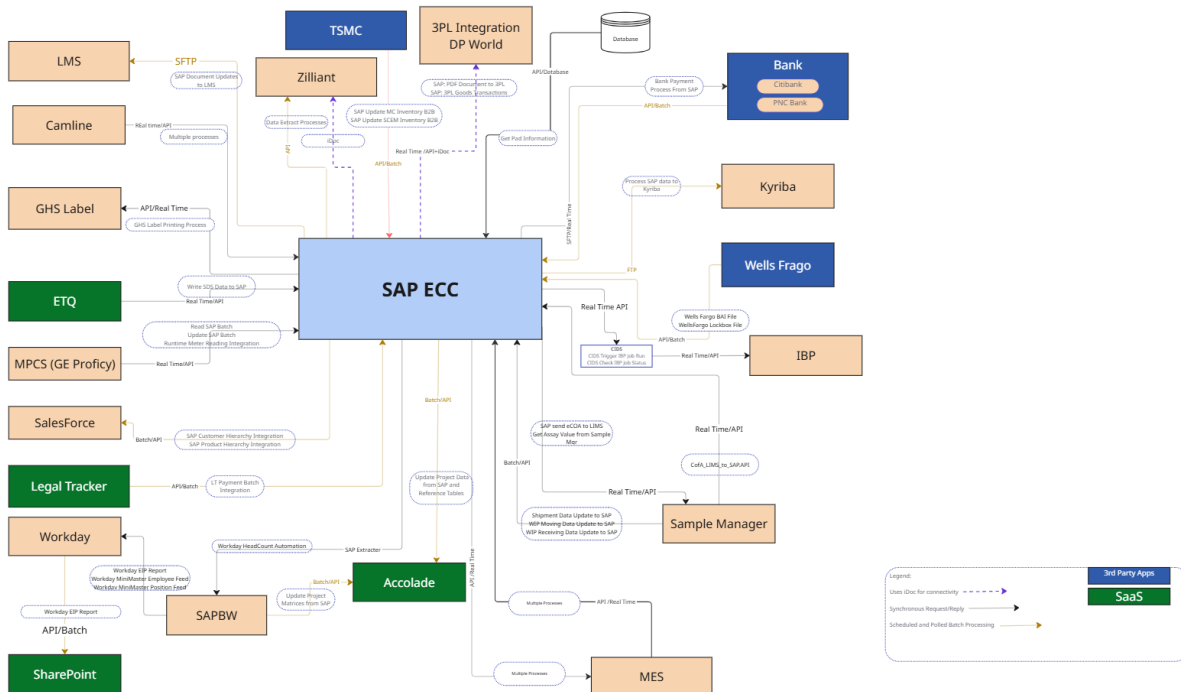
The following are the key considerations when selecting an integration pattern:

- **Transactional Integrity:** Writes must commit atomically; retries must not create duplicates.
- **Latency vs UX:** Some journeys require immediate confirmation (e.g., order number); others prefer decoupling for scale.
- **Availability & Resilience:** SAP availability/maintenance windows cannot block the caller's business flow.
- **Security & Compliance:** Strong authorizations, least privilege, audit trails, PII/financial data constraints.
- **Throughput & Volume:** Large/bursty loads (promotions, financial close) need queueing, batching, and back-pressure.
- **Operational Observability:** End-to-end correlation, traceability, error diagnostics, and replay.
- **Change Management:** SAP API versions and custom objects evolve. Avoid tight coupling and brittle integrations.

Why to use Integration Patterns

- Reduce cost through consolidation of functionality
- Agility through solutions based on a set of services that supports restructuring and reconfiguration of the business processes
- Time-to-market through business-aligned solutions that allow IT to rapidly support and implement
- Alignment between IT and business goals, enabling the business to associate capabilities with its strategic plan, leading to both sustained agility and re-use over time

AS-IS SAP Integration Architecture Diagram



Legend:

Synchronous Request/Reply

Scheduled and Polled Batch Processing - - - - -

Uses iDoc for connectivity

Architecture Principles

This section must be filled out for projects that have an architectural tier of one, two, or three.

Enterprise Principles

- Loosely coupled
- Reusability
- Platform standardization

Integration Principles

- API-first
- Use standard SAP APIs and events

Security-by-Design Principles

- Least privilege
- Zero trust

High Level Integration Solution Patterns

Quick Guide on When to use and When not to use Boomi for application integration:

Solution	When to Use	When Not to Use
Boomi	- A continuous business process - Synchronous (APIs/ other). Connecting to standard APIs/ pre-built connectors (SAP), mapping, orchestration - Pass messages or data between systems to perform functions through API - Synchronize business processes between multiple systems - When business events take place, automation is needed to notify or perform actions in one or more applications - Automation of repeated tasks that have clear business rules and requirements - Systems adhere to Entegris security standards and provide a way for us to connect May require a combination of technology - Data needs to be heavily manipulated before integration can occur - Manual input or intervention is required for successful integration - A user interface is required for integration - Automation of repeated tasks that have fuzzy or complex business logic - Use Boomi's SAP adapter for standardized/ typical interfaces - Use BTP to build out APIs/ other interfaces that Boomi consumes for complex custom interfaces	Asynchronous (store & forward) Most likely not the right solution when: - Data needs heavy manipulation through ETL (Extract, Transform, and Load) - Data extractions are large in volume and may run frequently - Applications do not allow for standard integration methods (API, SFTP, etc.) - Integration that is only required to run infrequently (1x a year)

Pattern Selection Matrix (Guidance)

Use this matrix to quickly shortlist the appropriate integration pattern for SAP write back with Boomi as the middleware between external systems and SAP.

Scenario Cue	Data Freshness	Payload	Connectivity	Direction	Recommended Pattern (Boomi → SAP)
Real-time UI or process requiring immediate SAP response	Real-time	JSON/XML	API (SAP ECC)	Inbound/Outbound	Synchronous API (Request/Reply) via Boomi
Scheduled or periodic updates to SAP	Hourly/Daily	Flat file / JSON/XML	API	Outbound	Batch Processing (SFTP/API) via Boomi
SAP-triggered outbound integrations needing immediate response	Real-time	JSON/XML	API/iDoc	Outbound	Synchronous API (Request/Reply) via Boomi
Data loads where SAP availability may vary	Eventual	JSON/XML / File	API	Outbound	Batch Processing with Retry (Boomi)
Real-time inbound calls from external systems requiring SAP data	Real-time	JSON/XML	API → Boomi	Inbound	Synchronous API (Pass-through to SAP)
Event-triggered updates to SAP where caller should not wait	Near real-time	JSON/XML	Message queue This option could be considered in future. Currently not in use	Outbound	Asynchronous Integration (Queue/Event-Driven inside Boomi)
High-volume uploads where immediate response is not required	Scheduled	File (CSV, XLS, Flat)	SAP Datatrans/API	Outbound	Batch File/API processing via Boomi
SAP → external systems where acknowledgement needed	Real-time	JSON/XML	API	Outbound	Synchronous API via Boomi

Scenario Cue	Data Freshness	Payload	Connectivity	Direction	Recommended Pattern (SAP → Boomi)
SAP → external systems (reports, extracts)	Scheduled	File (CSV, Flat)	API	Outbound	Batch File Processing via Boomi Note: SAP publishes scheduled extracts to Boomi via an outbound API. Boomi then manages downstream routing and delivery (SFTP, API, etc.), reducing SAP coupling and improving scalability and governance.

Integrating SAP with another System

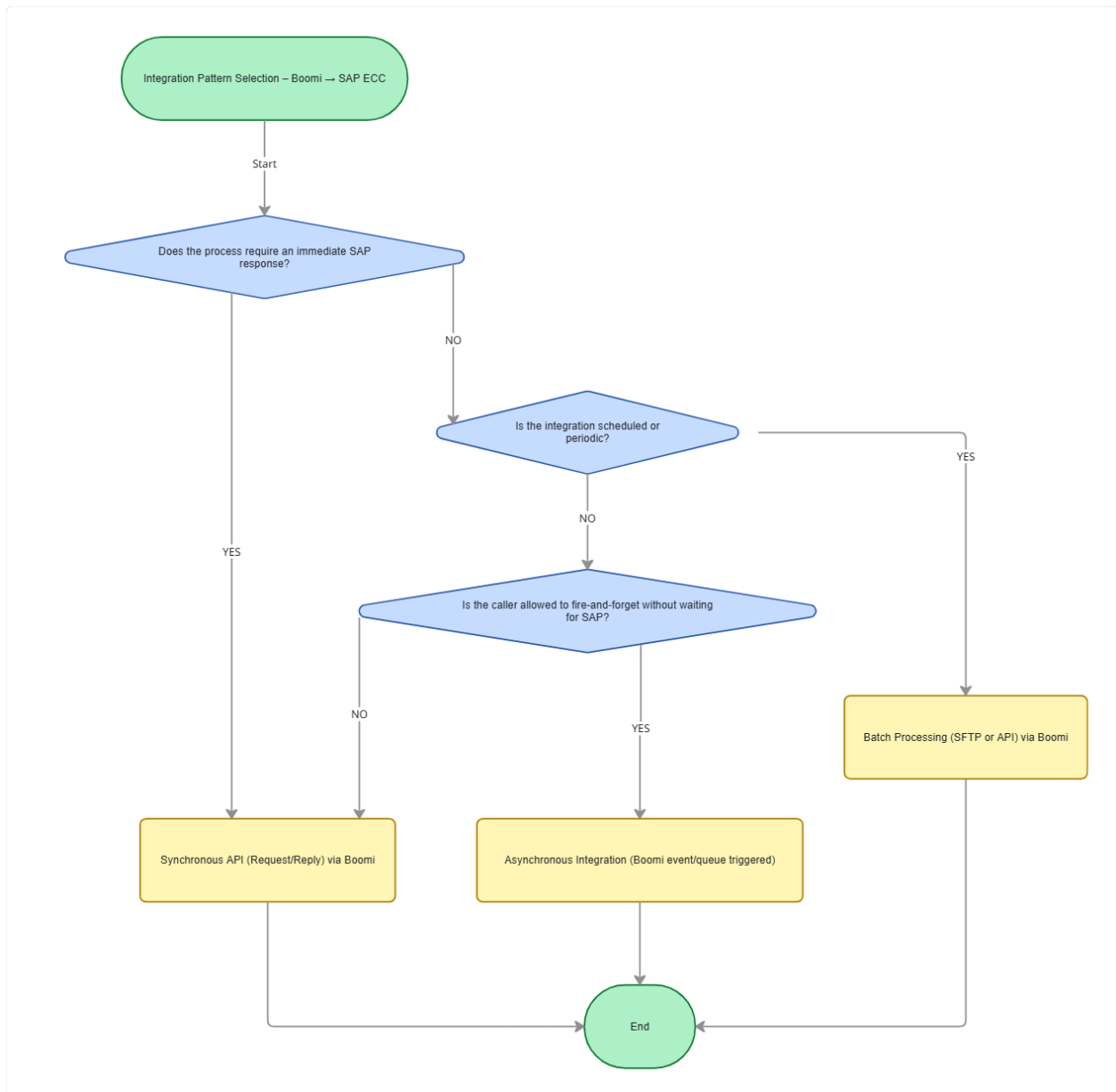
Type	Timing	Direction	Preferred Pattern (Boomi → Target)	Use When
Process Integration	Synchronous	SAP → Other	Synchronous API (Request/Reply)	SAP must trigger an action and get an immediate outcome/ack back.
Data Integration	Asynchronous (Scheduled)	SAP → Other	Batch Processing (SFTP/API)	Periodic extracts/reports; target doesn't need real-time; small payloads or where business logic is required. Note: Boomi is not to be used for high volume batch processing, core transaction processing,
Process	Asynchronous (Event/Schedule)	SAP → Other	Asynchronous (Boomi event/queue triggered)	Fire-and-forget from SAP or Boomi; resilient delivery with retry; target can process later.

Integrating Another System with SAP

Type	Timing	Direction	Preferred Pattern (Source → Boomi → SAP)	Use When and when not to use
Process Integration	Synchronous	Other → SAP	Synchronous API (Request/Reply)	Caller needs an immediate SAP result/confirmation.
Data Integration	Asynchronous (Scheduled)	Other → SAP	Batch Processing (SFTP/API)	Periodic loads, real-time need, small loads Periodic extracts/reports; target doesn't need real-time; small payloads or where business logic is required.

				Note: Boomi is not to be used for high volume batch processing, core transaction processing. For large loads Aecorsoft is used currently
Process	Asynchronous (Event/Queue)	Other → SAP	Asynchronous (Boomi event/queue triggered)	Fire-and-forget updates; caller shouldn't block; SAP availability may vary; rely on Boomi retry/reprocessing.

The following diagram explains **when to use or avoid** each core enterprise integration pattern. It provides guidance for architects to evaluate design trade-offs related to performance, SAP constraints, SLAs, and consumer expectations.

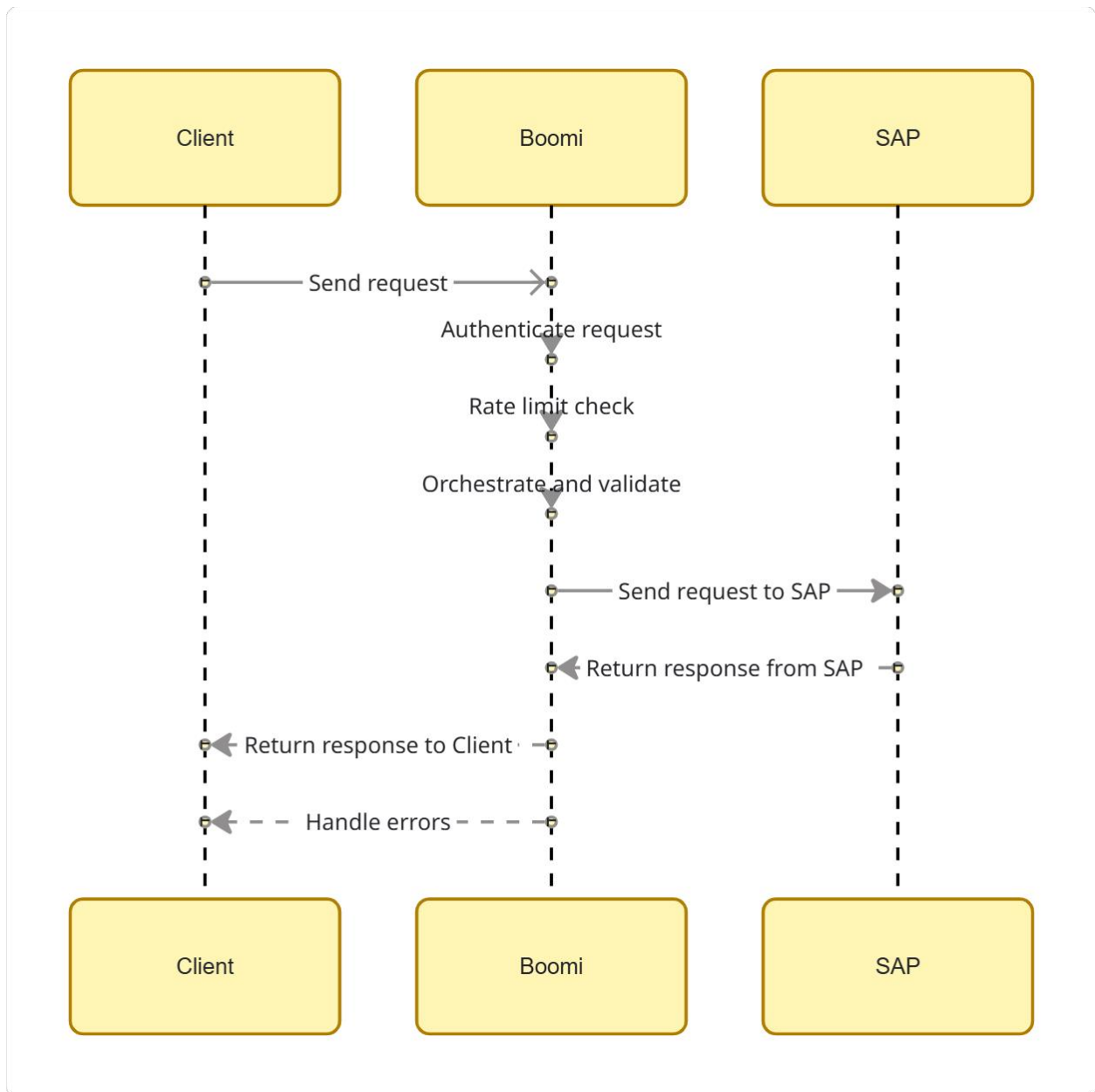


Canonical SAP Integration Patterns

Synchronous API (Request/Reply)

Area	Description
Context	A consumer needs SAP data or action with immediate confirmation (e.g., pricing/availability, validation calls).
Problem	How do we provide low-latency, reliable, secure access to SAP capabilities from external apps?
Solution	• Expose/consume APIs via Boomi with request/response. • Enforce authentication, authorization, schema validation, idempotency, and strict timeouts.
Benefits	• Fast user experience • Predictable contracts • Strong Governance
Considerations	• Requires strict SLAs and rate limiting • Back pressure needed under load

Diagram

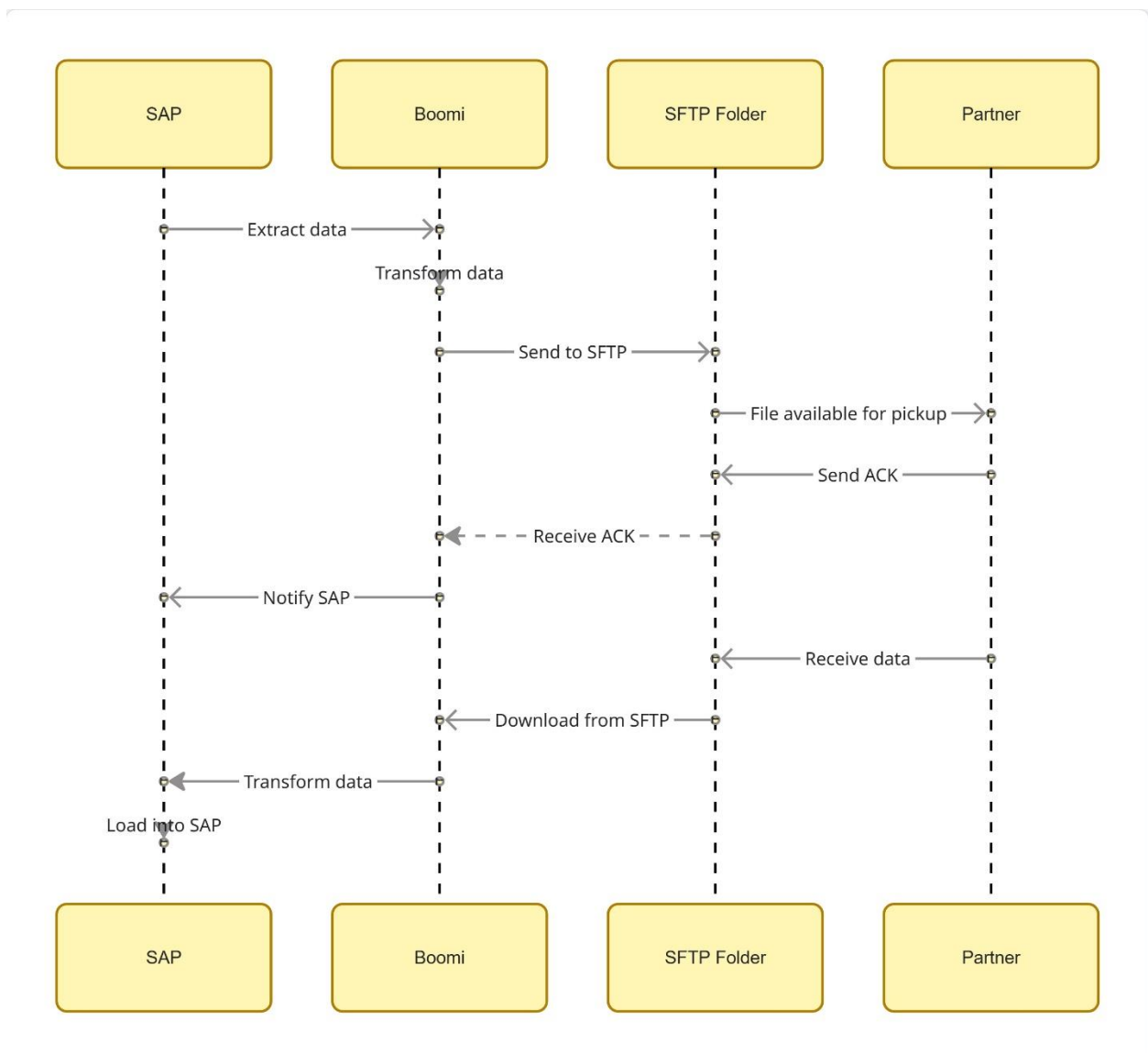


Batch SFTP (File-based Extract/Load)

Area	Description
Context	Partner systems exchange files at scheduled intervals when APIs are unavailable or impractical
Problem	How do we move larger data sets reliably on a schedule with strong operational controls?

Solution	• Use Boomi to generate/ingest delimited files over SFTP with naming/versioning, encryption, and acknowledgements • Implement reconciliation and reprocess flows. • Avoid ad-hoc layouts.
Benefits	• Simple and robust • Handles large volumes
Considerations	• Not real-time • Operational load for file management

Diagram



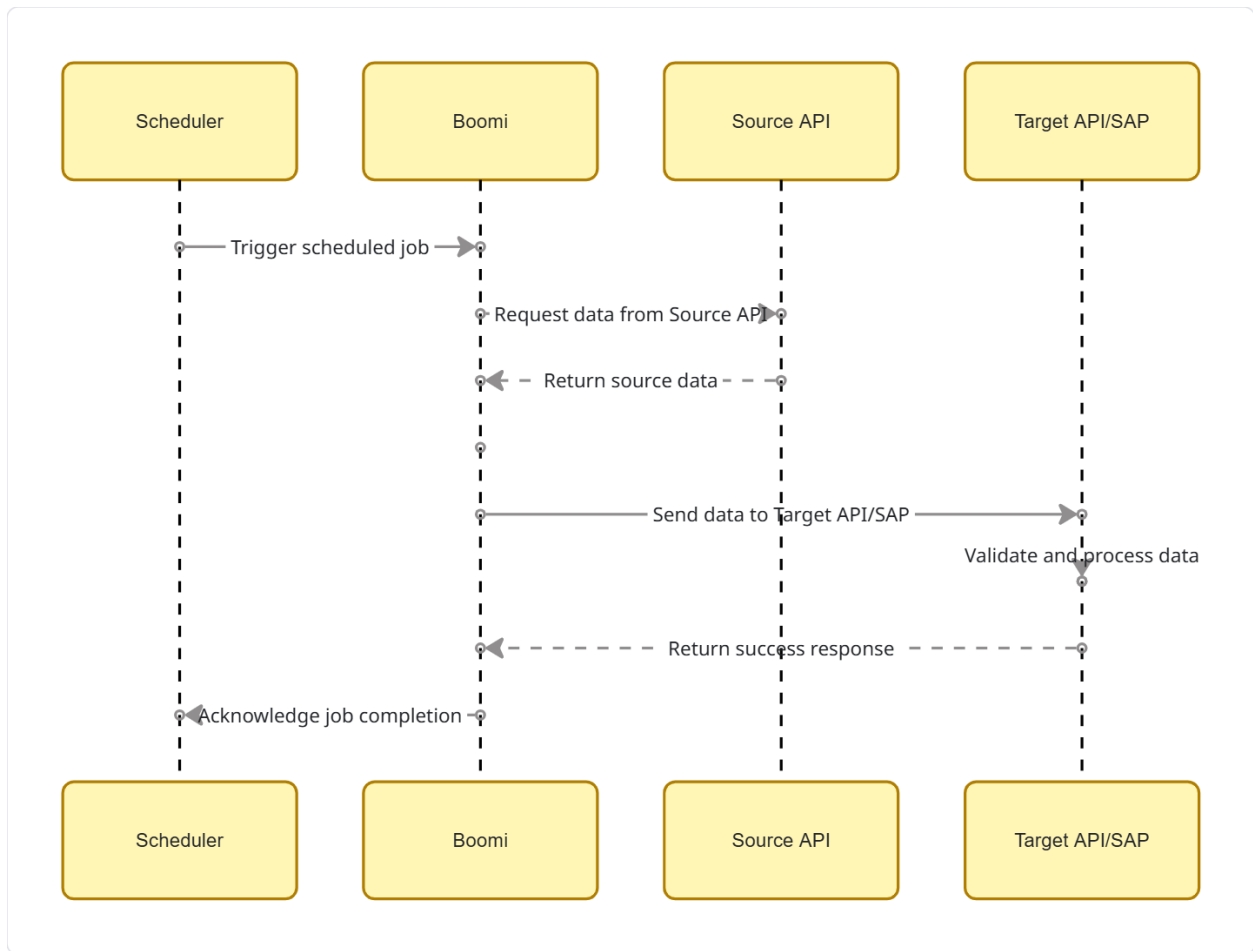
Batch API (Scheduled Extract/Load)

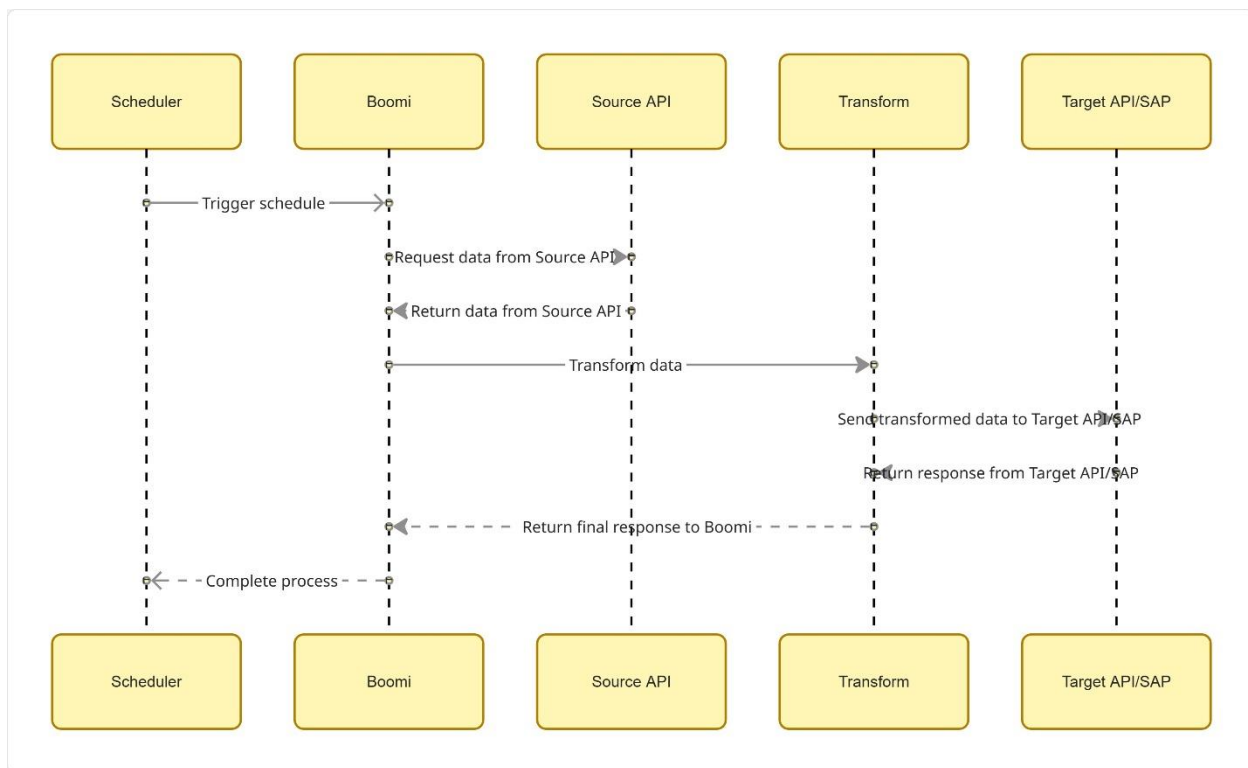
Area	Description
------	-------------

Context	Systems provide APIs but business is batch-oriented (e.g., nightly sync).
Problem	How do we orchestrate scheduled loads without file landings while preserving observability?
Solution	Use Boomi to call external APIs on a schedule, handle pagination/delta windows, and write back to SAP or targets; avoid synchronous fan-out during peaks.
Benefits	• No file ops • Clear contracts
Considerations	• API throttling windows • Need careful delta and error handling

Diagram

=

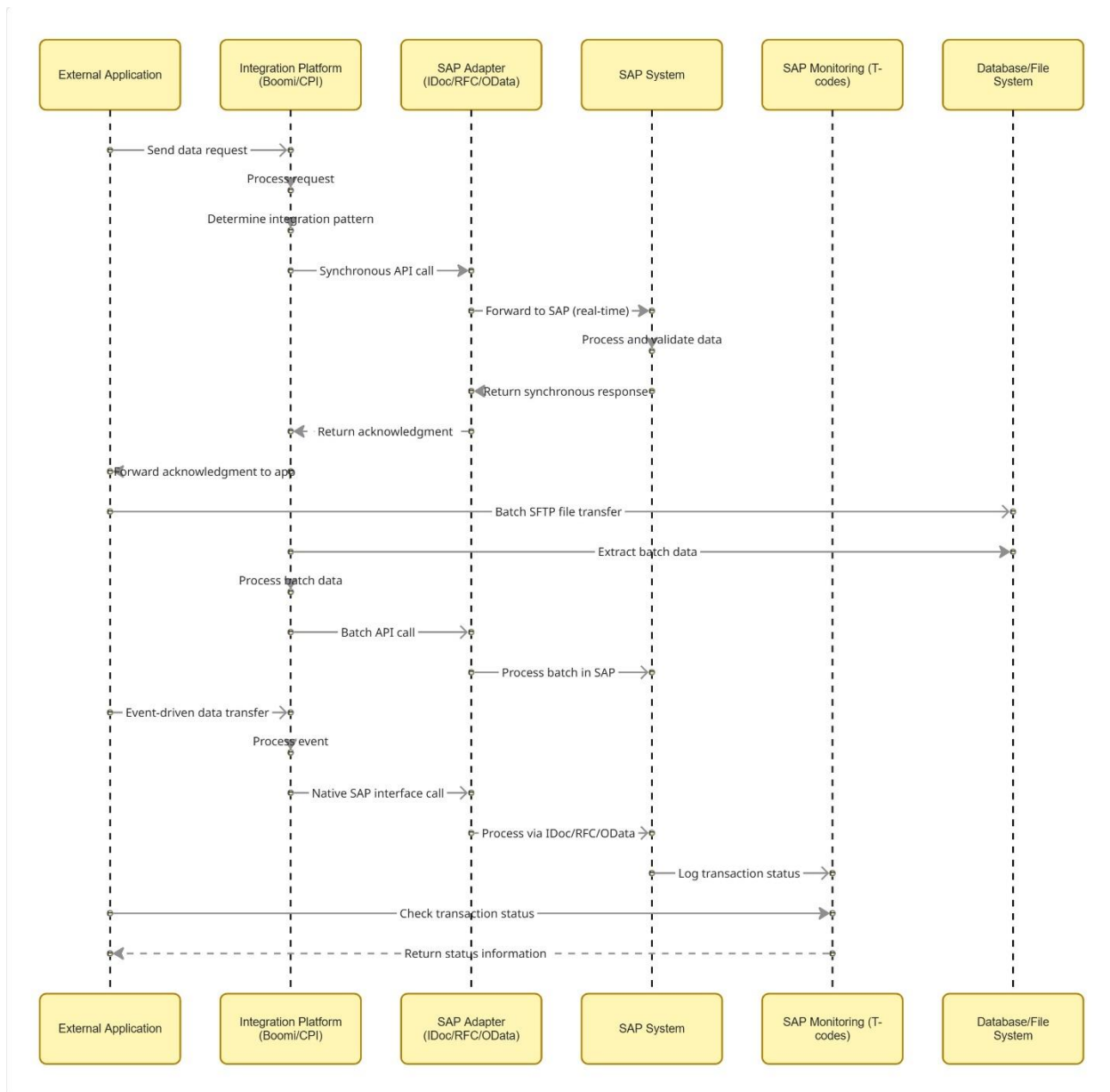




SAP IDoc / RFC / OData (SAP-native)

Area	Description
Context	Deep SAP integration where SAP standard interfaces exist and need reliable messaging semantics
Problem	How do we leverage SAP-native integration while maintaining governance and resilience?
Solution	Adopt IDoc for document-style async, RFC/BAPI for remote function calls, or OData for RESTful access; use Boomi/CPI or PO as mediation; standardize change pointers/output types; avoid custom Z endpoints unless necessary.
Benefits	• Standardized supportability • Rich monitoring in SAP
Considerations	• Requires SAP expertise • Transport governance overhead

Diagram:

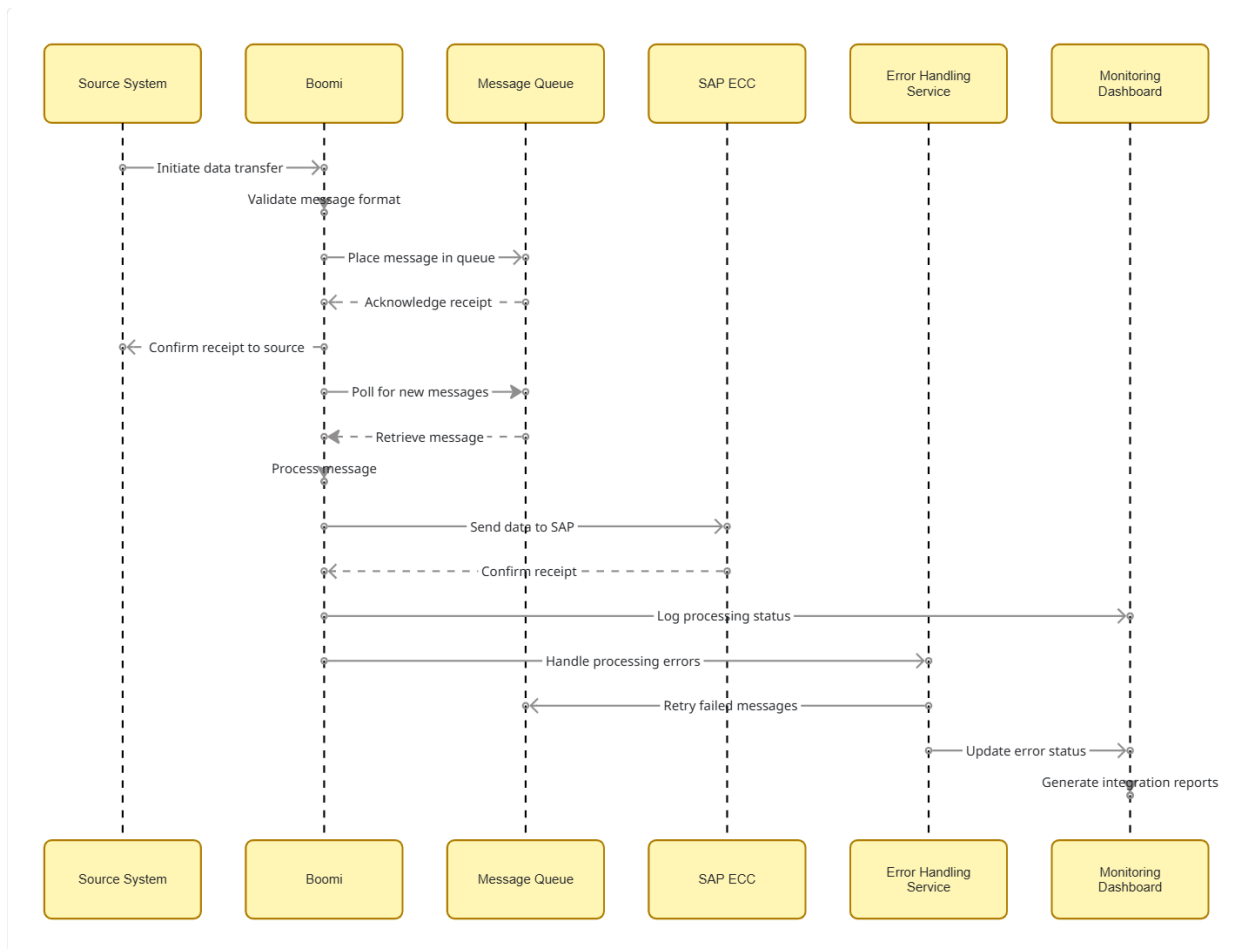


Asynchronous Pattern

Area	Description
Context	Asynchronous SAP write-back or event-triggered processing where the source system should not wait for SAP to respond. Boomi acts as the mediation and orchestration layer, providing durable execution, retry, and scheduling.
Problem	How do we perform SAP updates reliably when real-time response is not required, or when SAP availability is variable, while still ensuring

Area	Description
	governance, resilience, and auditability through Boomi?
Solution	Use Boomi asynchronous processing patterns such as message queuing, event-triggered integration, scheduled execution, or decoupled process routing. Boomi interacts with SAP only through standard SAP ECC APIs (synchronous RFC/OData via Boomi connector), but the triggering pattern is asynchronous fire-and-forget, retry-based, or scheduled.
Benefits	• Decouples source systems from SAP availability • Fail-safe delivery with retry, logging, and alerts in Boomi • Standardized operational monitoring • Supports low to moderate data volumes
Considerations	• Requires Boomi process design expertise for retry, queueing, and reprocessing • SAP is still invoked via synchronous calls under the hood, so throughput needs tuning • Requires governance on scheduling, throttling, and error-handling frameworks

Diagram:



Tips and Best Practices for Integration

- Use canonical data models where possible to keep mappings centralized
- Define clear SLAs (latency, throughput)
- Error handling contracts (DLQ, retries, notifications)
- Secure transport (TLS/SFTP), rotate keys/certs, and classify data (PII/PCI)
- Design for idempotency and exactly once effects where feasible; otherwise support compensation
- Implement run-time observability such as correlation IDs, structured logs, metrics, and dashboards
- Establish transport and content versioning, plan backward compatibility windows

Separate configuration (endpoints, secrets) from code